

Tu Ford te avisa antes

Predicción temprana de la degradación del filtro de partículas en los diésel conectados de Ford

Informe final del equipo SOG · Gonzalo · Santino · Octavio · 2 de octubre de 2026

Resumen ejecutivo

- **El problema.** El filtro de partículas (DPF) de los diésel se tapa según cómo se usa el auto, y hoy Ford se entera tarde: cuando se prende el testigo o el auto ya está en el taller. El desafío pide anticiparlo con la telemetría del auto conectado, con el mayor margen posible.
- **Lo que encontramos.** La telemetría deja huella: comparando contra sanos del mismo mercado, los autos que van a fallar pasan cada vez más tiempo en ralentí y con el motor frío. Pero los datos traían trampas que hacían que cualquier modelo pareciera mejor de lo que es (cohortes muestreadas en períodos distintos, una fecha del evento posterior al taller, un contador que se apaga). Las corregimos en un pipeline declarativo y trazable.
- **Lo que hicimos.** Cada 500 km, un modelo lee los últimos 1.000 km del auto y estima si falla en los próximos 3.000, sin ver nunca los 500 km previos a la falla. El finalista es una red recurrente (GRU) que lee la secuencia de viajes y de señales del filtro, más país, motor y modelo, en un ensamble de tres semillas. Un circuito convierte cada alerta en un consejo al conductor o en un llamado del concesionario, redactado por agentes de IA y controlado por un verificador en código.
- **Cuánto detecta [test].** En 111 autos que nunca vio (103 con datos, 32 fallados), avisa a ~30 % de las fallas con 5 % de autos sanos con falsa alarma (10 de 32, donde el azar da 7 %), a ~40–50 % con 10 % y a ~50–60 % con 20 %. Con el umbral fijado antes, en desarrollo, las falsas alarmas reales quedan en 5,8 · 8,9 · 18,2 %.
- **Con cuánto margen [test].** La primera alerta llega con una mediana de ~4.600 km, unos 3 meses, antes de la falla registrada.
- **Cuánto cuesta.** Sin sensores nuevos y en CPU: ~USD 0,07–0,10 por auto y por año para un millón de autos (estimación propia, a validar con Ford).
- **Lo que no prometemos.** No le gana con evidencia a la regla “país × motor” (+27,3 puntos, IC95 [−0,8; 46,1]). Sabe sobre todo *qué auto* va a fallar y poco *cuándo*. Con 32 fallas los intervalos son anchos y el test se leyó más de una vez: por eso citamos rangos. La muestra está enriquecida en fallas, así que la prevalencia real se mide en un piloto. No predecimos la fecha exacta.

Índice

Resumen ejecutivo	1
Cómo leer este informe	1
1. Descripción general del desafío	2
2. Contexto del proceso	3
3. Definición del desafío	3
3.1. La pregunta: cortes cada 500 km con gap de blanking	3
3.2. Cómo medimos, y por qué cambiamos de métrica	4
3.3. Impacto operativo	4
4. Datos técnicos relevantes	5
4.1. Las tablas, los sistemas y las restricciones	5
4.2. Trampas, limpieza, faltantes y atípicos	5
4.3. El universo y el holdout	5
4.4. Las features	6
4.5. Qué dicen los datos	6
4.6. Consistencia y trazabilidad	7
5. Expectativas de solución	7
5.1. Objetivo de mejora: anticipar con el mayor margen posible	7
5.2. Criterios de evaluación	10
6. Información complementaria	12
A. Reproducibilidad	13
B. Auditorías de leakage y el techo de cohorte	13
C. Todos los modelos en test y en desarrollo	14
D. Diccionario de features	15

Cómo leer este informe

El informe sigue las siete secciones del documento de Ford. En cada una contamos qué entendimos, qué encontramos en los datos y qué hicimos.

- **[test]:** medido sobre los 111 autos del holdout, sorteados antes de mirar ninguna feature contra la etiqueta (103 con datos: 32 fallados y 71 sanos). Cada modelo se entrenó con todo el conjunto de desarrollo. **[dev]:** validación cruzada agrupada por vehículo sobre los 446 autos de desarrollo. La demo, la explicabilidad y las auditorías son de dev.
- **Detección al X %:** de los autos que fallan, cuántos reciben una alerta antes de la falla con un umbral fijado para que solo el X % de los sanos tenga una falsa alarma. Una alerta son dos cortes seguidos sobre el umbral.

- **Azar:** el mismo puntaje permutado entre filas, conservando el largo del historial de cada auto. **Celda mercado × motor:** la tasa de fallas de la celda del auto como puntaje fijo, sin mirar ningún viaje. Es el piso que cualquier modelo tiene que superar.
- **Nada de accuracy:** con ~6 % de filas positivas, “nadie falla” da 94 %. El PR-AUC por fila va al Anexo B, al lado del techo de cohorte.

Tabla 1: Los criterios de Ford (§5.1 y §5.2 del desafío) y dónde se responden.

Criterio	Qué mostramos	Dónde
Objetivo 5.1: anticipar con el mayor margen	~4.600 km (~100 días) de mediana en test, con 500 km ciegos antes del evento; el puntaje sube hacia el evento	§3.1, §5.1.4
Viabilidad técnica	Medido en un holdout congelado; GRU de ~3.900 parámetros en CPU; demo funcionando; plan por etapas con piloto en sombra	§5.2.1
Costo	~USD 0,07–0,10 por auto y por año, sin sensores nuevos	§5.2.2
Carácter innovador	Una métrica que separa <i>qué auto</i> de <i>cuándo</i> ; red recurrente sobre la ventana; IA generativa con verificador en código; la perilla de falsas alarmas	§5.2.3
Calidad del tratamiento de datos	Trampas de formato y de muestreo corregidas; atípicos con topes físicos; faltantes estructurales tratados como información; correcciones declaradas, holdout congelado y auditorías de leakage	§4, Anexo B
Claridad para mostrar los datos	Curva de detección contra falsas alarmas con el azar dibujado; figuras con fuente; demo con tres pantallas y perilla	§5.2.4
Explicabilidad	En qué se apoya la GRU (permutación) y qué se le dice al conductor (comparación con la flota sana, filtrada por la física del DPF)	§5.2.5
Pitch	La misma historia y los mismos números, en siete bloques y 29 minutos (docs/pitch/)	§5.2.4

1 Descripción general del desafío

Título. Ford pide una metodología de IA para predecir eventos de degradación de la eficiencia de combustión e ingreso de oxígeno al motor, a partir de datos de vehículos conectados.

Qué entendimos. En un diésel, el componente que se degrada por el uso y afecta a la vez la combustión y el flujo de gases es el filtro de partículas (DPF). El filtro atrapa el hollín y lo quema para limpiarse (regenera), pero para eso necesita temperatura: un tramo de ruta con el motor caliente. Con viajes cortos, la regeneración empieza y no termina, el hollín se acumula y, si sigue así, el auto entra en modo de protección y termina en el taller. Para el cliente, eso es un testigo que nadie le explicó y la sensación de que nadie le avisó.

Qué encontramos en los datos.

- **El evento es del filtro, con otro nombre.** *TripSummary* no trae las columnas del DPF del anexo, pero sí *AirRegenerationStart/End* y *AirFilterStart/End*, que el anexo no menciona. Son la foto, al inicio y al final de cada viaje, de la acumulación (0–100) y de los nueve niveles de mensaje del filtro que trae *DynamicInformation* (coinciden en el 99,8 y el 99,97 % de los pares). Ford no lo confirmó, así que hablamos de “nivel de acumulación del filtro”.
- **Es saturación por uso, no desgaste.** El evento cae a una mediana de ~12.800 km, y la ceniza es un fenómeno de más de 100.000 km. Buscamos una carga irreversible medible (la premisa de un proceso de degradación tipo gamma) y no la hay. Eso orientó todo hacia los hábitos de manejo y el estado reciente del filtro.
- **No todo “uso indebido” se sostiene.** Comparando contra sanos *del mismo mercado*, lo que crece hacia el evento es estar detenido con el motor en marcha y no llegar a temperatura. La velocidad

y la fracción de viajes cortos no distinguen dentro del mercado: con la primera entrega, la velocidad parecía la mejor señal y era Colombia contra Chile.

Qué hicimos. Un modelo de deep learning que lee la secuencia de los últimos 1.000 km del auto y un producto que convierte cada alerta en una acción. Al conductor no le hablamos de “uso indebido” ni de fallas: le hablamos de hábitos que puede cambiar, comparados con autos sanos de su zona.

2 Contexto del proceso

Qué entendimos. La telemetría llega a la nube de forma continua, pero la degradación se identifica una vez manifestada, en un service o cuando el cliente la reporta. El desafío se ubica en la posventa.

Qué encontramos en los datos: el proceso reactivo está escrito en la telemetría. La fecha registrada del evento llega después del taller. En la primera entrega coincidía con la intervención (el cambio de aceite se concentraba en ± 3 días). En la entrega v2, la fecha de los mismos autos cae una mediana de 14 días después, y en las dos semanas previas ya aparecen el cambio de aceite y el ralenti largo de la regeneración forzada (1,5–1,8 veces la base). Después del evento, además, el uso cambia: la fracción de viajes bajo régimen térmico baja en el 72 % de los autos.

Qué hicimos. Si el modelo viera el taller, haría la detección reactiva que Ford ya tiene. Por eso la referencia del evento es la **fecha registrada menos 21 días**, el modelo nunca ve los **500 km previos** a esa referencia (§3.1) y nada posterior al evento entra a ninguna ventana. En la posventa, la alerta entra a un circuito que termina en una acción (figura 1): con un hábito para nombrar, un consejo al conductor en la app; sin hábito, un llamado del concesionario; si el riesgo sigue alto dos revisiones después, un escalamiento. El gerente de posventa recibe cada lunes una bandeja priorizada y redactada, lista para aprobar.

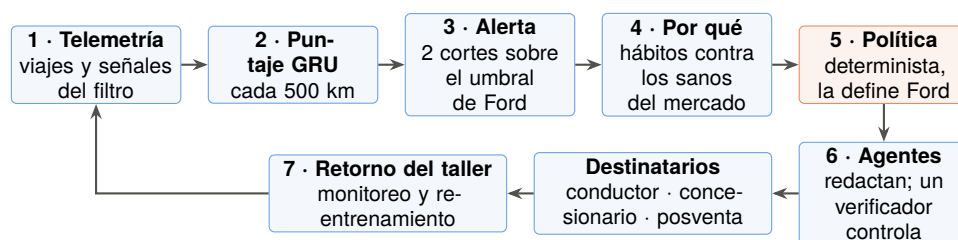


Figura 1: El circuito de posventa: el modelo decide, el agente comunica y el código cuida lo que se dice.

Fuente: docs/pitch/presentacion-contenido.md bloque 4; feat/demo-gru:PRODUCT.md.

3 Definición del desafío

Qué entendimos. Ford pide anticipar *el momento* en que un vehículo se aproxima a la degradación, a tiempo para actuar antes de que el cliente lo note. Son dos exigencias que tiran para lados distintos: *anticipar* pide no mirar los últimos kilómetros, que es donde está la señal más fuerte, y *el momento* pide saber *cuándo*, no solo *qué auto*. Lo más difícil fue convertir eso en una pregunta que no se pudiera ganar con atajos.

3.1 La pregunta: cortes cada 500 km con gap de blanking

Armamos un panel con una fila por (vehículo, corte), con cortes cada $\Delta = 500$ km. En cada corte el modelo ve los últimos $W = 1.000$ km y responde si el evento cae entre $G = 500$ y $G + H = 3.500$ km después (figura 2).

- $G = 500$ km es la regla antirreactiva: sin el gap, el modelo memoriza los avisos de “filtro sobrecargado” de los últimos kilómetros. $W = 1.000$ km: ventanas de 2.000 y 3.000 km daban lo mismo y costaban eventos. $H = 3.000$ km pone los positivos donde el perfil alineado muestra

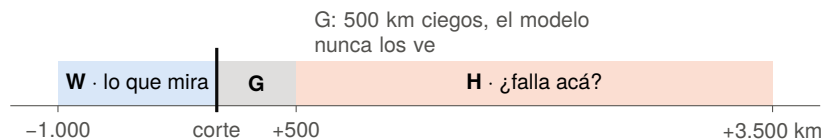


Figura 2: Ventana W hacia atrás, gap G y horizonte H hacia adelante. La etiqueta vale 1 si el evento cae en H.

Fuente: `configs/data/panel_v2.yaml`; `docs/memoria/decisiones.md` (18-09).

señal (§4.5).

- **El evento se ordena por tiempo, no por km** (dispersión del logaritmo 0,38 en edad contra 0,91 en odómetro): la anticipación se reporta en km y en días.
- **Sanos emparejados por odómetro y por mes:** sin emparejar por mes, tres columnas de calendario subían el ROC de 0,67 a 0,76; emparejando, de 0,57 a 0,60 (primera entrega). El calendario no es el auto.
- El panel de desarrollo tiene 11.703 filas de 426 autos con cortes (135 fallados) y 682 positivas (tasa 0,058).

Reglas que no se negocian: gap de blanking; split agrupado por vehículo, con folds estratificados por etiqueta, mercado y motor y CV repetida tres veces; holdout de 111 autos congelado antes de mirar ninguna feature contra la etiqueta; features solo hacia atrás, con imputación y escalado ajustados en el train de cada fold; auditorías de leakage obligatorias (Anexo B); y nada hardcodeado: cada corrida es un YAML versionado.

3.2 Cómo medimos, y por qué cambiamos de métrica

El techo de cohorte

Todas las filas positivas del panel están dentro de los cortes de autos que fallan. Un modelo que solo sabe “este auto es de los que fallan”, sin nada del *cuándo*, saca **PR-AUC 0,177 y lift 3,03×** (0,263 y 2,10× con la primera entrega). La meta inicial de lift (1,6–2×) se alcanzaba sin anticipar nada, y ningún modelo superó ese techo: **el PR-AUC por fila de este panel mide sobre todo qué auto.**

Fuente: `docs/memoria/decisiones.md`, cierre de F3 (20-09); `src/eval/metrics.py`.

Desde entonces la pregunta que decide es una sola: *de los autos que van a fallar, ¿a cuántos les avisamos a tiempo si solo permitimos una cantidad fija de falsas alarmas entre los sanos?* Para cada presupuesto (2 a 30 % de los sanos) el umbral es el menor que lo cumple, y una alerta son dos cortes seguidos sobre él. El criterio primario es la detección media al 5, 10, 15 y 20 %, y siempre se compara contra tres cosas: el **azar con el mismo historial** (con la primera entrega, bolsas asimétricas “detectaban” 10 % al 5 % sin ninguna señal), la **celda mercado × motor** (tres celdas concentran 117 de los 135 fallados de dev, y esa regla sola detecta 18 · 33 · 62 % al 5 · 10 · 20 % [dev]) y el modelo de referencia, con **bootstrap pareado por vehículo**. El umbral también se fija **fuera de muestra** para medir las falsas alarmas reales. Con unos 12 eventos por fold, cada candidato nuevo gastaba un *presupuesto de comparaciones*, y los importantes se preregistraron por escrito antes de medirlos.

3.3 Impacto operativo

Ford menciona satisfacción del cliente, costos de garantía, vida útil percibida y planificación de concesionarios. El producto responde así: el aviso llega con unos tres meses de margen [test], en lenguaje humano y con algo concreto para hacer; si el auto no iba a fallar, el conductor igual recibió un consejo útil. La bandeja semanal prioriza los casos para el concesionario, y la perilla deja que Ford elija cuántas alertas acepta. Cuántas fallas se evitan no se puede medir con estos datos: es la métrica central del piloto (§5.2.1).

4 Datos técnicos relevantes

Esta sección responde al criterio de *calidad del tratamiento de datos*: qué llegó, qué trampas tenía, qué descartamos y cómo quedó trazado cada paso.

4.1 Las tablas, los sistemas y las restricciones

Qué llegó. Seis CSV: las tres tablas del anexo, cada una partida en dos cohortes (fallados y sanos), en dos entregas. La segunda (26-09) trajo estática nueva, con ciudad de venta y país ISO, y viajes y señales nuevos de fallados. Los de sanos son byte a byte los de la primera. En total, ~2,5 millones de viajes y ~10 millones de señales.

Qué encontramos al contrastar con el anexo. *TripSummary* trae 25 de las 40 columnas documentadas: faltan GPS, elevación, presión de neumáticos, hollín de regeneración manual y autonomía. El DPF llega con otro nombre (§1). No hay VIN: la clave es un código anonimizado, y el join entre las tres tablas cierra. El evento no está marcado fila a fila: la fecha del evento existe en el 100 % de los fallados y en ningún sano. **La cohorte es la etiqueta**, y nunca entra como variable. El historial sobra para ventanas de varios miles de km (mediana de ~2.000 viajes y más de un año por auto en la primera entrega). Sin GPS ni elevación, las features de altura murieron; con la v2 probamos la altura de la ciudad de venta, que no se separa del cero (§4.5).

4.2 Trampas, limpieza, faltantes y atípicos

Cada una de estas trampas rompe algo en silencio: no hay error, solo un número mejor de lo que debería (tabla 2).

La imputación (mediana) y el escalado se ajustan con el train de cada fold, dentro del pipeline. Hay una pregunta que solo Ford puede contestar: entre las dos entregas salieron 278 fallados con la fecha por defecto, concentrados por mercado. No sabemos si eran fallas sin fecha o un error de la consulta, y eso explica en parte por qué la tasa cambia tanto entre países.

4.3 El universo y el holdout

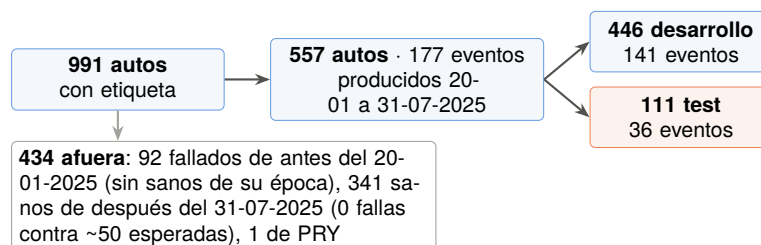


Figura 3: Del dataset al holdout. El test se sorteó una vez (semilla 42, estratificado por evento × mercado × motor) antes de mirar ninguna feature contra la etiqueta.

Fuente: docs/memoria/f9-universo-v2.md; configs/data/test_split.yaml.

Se conserva el período de producción donde se muestrearon las dos cohortes: un sano de octubre de 2025 no es un negativo verificable si de esa época no se registró ninguna falla. El equipo propuso sacar el recorte para no tirar 433 autos, y lo probamos de punta a punta: con ellos, los modelos ordenaban *peor* a los autos comparables (survival stacking, de 0,571 a 0,471 dentro de mercado × motor), porque aprendían la fecha. Volvimos a la ventana. Con la v2 el holdout se volvió a sortear: 36 de sus 111 autos estaban en el desarrollo de la primera entrega, y el test se reporta también sin ellos.

La segunda entrega cambió varias conclusiones, y lo contamos: la velocidad dejó de ser señal dentro del mercado; apareció una trayectoria hacia el evento que antes no se veía; la “ventana de calendario del registro” resultó ser de la extracción vieja; el motor ENG_3, que nunca fallaba, pasó a tener 47 eventos (46 en Brasil); y el finalista de la primera entrega (survival stacking con esa

Tabla 2: Lo que encontramos en los datos y qué hicimos con cada cosa.

Problema	Evidencia	Tratamiento
Muestreo y etiqueta		
Dos listas muestreadas en periodos de producción distintos	En la v2, antes del 20-01-2025 solo hay fallados (93) y después de julio de 2025 solo sanos (341, contra ~50 fallas esperadas). Con esos autos, la fecha de producción sola separa las cohortes con AUC 0,87 dentro del mercado	Universo = período común, 20-01 a 31-07-2025 (§4.3)
La fecha registrada es posterior al taller	Cambio de aceite y ralentí largo en las dos semanas previas; la fecha de la primera entrega caía 14 días antes (mediana)	Referencia del evento = fecha – 21 días
La tasa por país depende de la lista	Va de 0 % (PER) a 48 % (COL), en el orden inverso de cuántos fallados sin fecha sacó Ford de cada mercado	Toda comparación, dentro del mercado; todo AUC, también dentro de mercado × motor
13 autos con dos códigos	Mismos viajes bajo dos identificadores (Jaccard 1,0 sobre fecha y odómetro)	Se colapsan antes de cualquier split: si no, un auto cae en train y en validación
Formato de la v2	Columna del evento renombrada; producción de fallados contada desde otro origen (+538 días); una fila por evento; filas repetidas; extracción de fallados diez días más larga (16.911 viajes que solo puede tener un fallado)	Corrección al leer, declarada en <code>raw_sources.yaml</code> y probada en <code>check_setup.py</code>
Después del evento el uso cambia	Bajo régimen baja en el 72 % de los autos y la velocidad sube en el 79 %	Nada posterior al evento entra a ninguna ventana
Columnas, faltantes y atípicos		
El contador de regeneraciones se apaga	Cero marcadores desde el 25-05-2026 en toda la flota; su correlación con la etiqueta (0,317) era calendario	Solo auditoría; regeneraciones = caídas de la acumulación de al menos 15 puntos (con 5, el saldo no seguía la acumulación: correlación 0,005 contra 0,45)
Velocidad imposible y nula	Hasta 135.600 km/h; nula <i>exactamente</i> en los viajes de 0 km (33 % de las filas)	Recalculada como km/duración con tope de 200 km/h; el ralentí es una señal propia, no un faltante
Temperaturas, duraciones y km extremos	–73 °C, viajes de 20 días y de –230 km (< 0,05 % cada uno)	Topes físicos fijos, no cuantiles; los viajes con km negativos o sin odómetro se descartan
Columnas que no son lo que dicen	El combustible supera 100 en el 10,5 % de los viajes; la vida del aceite no cambia dentro del viaje; la acumulación llega a 100, no a 95	Combustible como índice relativo; pendiente del aceite en la ventana; saturado = nivel ≥ 95
Fechas en dos formatos	Sin declararlo, pandas anulaba en silencio 4.464 inicios de viaje y 18.124 señales	ISO 8601 declarado en el loader
Odómetro de señales nulo	11 % en la primera entrega, concentrado en 9 autos; 0,5 % en los fallados v2	La ventana se ubica por el odómetro de los viajes, casi completo
Tramos de 50 km sin viajes	Huecos en la secuencia	Canal de cobertura y arrastre del último valor: un tramo vacío no es “cero ralentí”

Fuente: `f9-entrega-v2.md`, `f9-universo-v2.md`, `f9-eda-v2.md`, `f1-*.md`, `f2-*.md`; `configs/data/features_v1.yaml` (`clip`, `thresholds`).

ventana, ~15–17 % al 5 %) perdió su corrección.

4.4 Las features

La ventana de 1.000 km llega al modelo de dos formas (diccionario completo en el Anexo D): **53 features agregadas** para los modelos tabulares y de supervivencia, en cuatro familias (térmica y trayectos cortos, regeneración, uso y severidad), y **la secuencia** para los recurrentes: 20 tramos de 50 km × 15 canales, 9 de las señales del filtro y 6 de los viajes, más país, motor y serie. **Quedan solo para auditar** lo que mide el calendario o la cohorte y no el auto: fechas de producción y de venta, temperatura ambiente (los fallados viven en lugares más fríos que los sanos de su mercado) y el contador cortado. Motor y serie entran en la v2, pero aportan la tasa de su celda y no orden entre autos: por eso todo se reporta también dentro de mercado × motor.

4.5 Qué dicen los datos

Todo es de desarrollo [dev] y compara contra sanos *del mismo mercado*: con cinco mercados de tasas tan distintas, una comparación agrupada infla lo que es país.

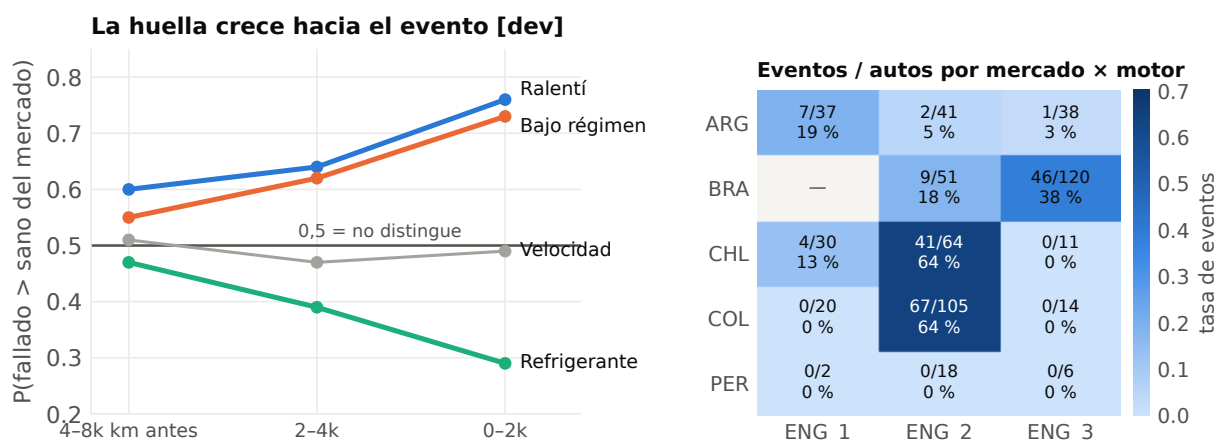


Figura 4: Izquierda: probabilidad de que un auto que va a fallar supere a un sano del mismo mercado y tramo de odómetro, según la distancia al evento. Derecha: eventos sobre autos por mercado × motor en el universo.

Fuente: docs/memoria/f9-eda-v2.md §E; f9-universo-v2.md §3.

- **La huella crece hacia el evento:** ralentí (0,60 → 0,64 → 0,76) y bajo régimen (0,55 → 0,62 → 0,73), y el refrigerante termina los viajes más frío. Lo que crece es estar detenido con el motor en marcha y no llegar a temperatura, no manejar distinto. Lejos del evento la señal es débil.
- **Se ve temprano:** con los primeros 60 días de uso tras la venta, el ralentí ordena autos con AUC 0,636 dentro del mercado. Es señal de *qué auto*.
- **El riesgo es de edad, no de km:** sube hasta los ~4 meses desde la venta y después queda en ~3–4 eventos por 100 autos-mes; el mes calendario no agrega nada ($p = 0,16$).
- **Tres celdas concentran casi todas las fallas** (Colombia y Chile con ENG_2, Brasil con ENG_3). Qué parte es física y qué parte es cómo se armó la lista no se puede separar con estos datos: es la pregunta más importante para Ford.

4.6 Consistencia y trazabilidad

Las correcciones de los crudos se declaran en YAML y las aplica el loader. Cada decisión vive en un solo lugar: el universo en `src/data/usable.py`, los clones en un YAML congelado, el split en `src/eval/splits.py`. El código falla si el panel trae un auto excluido. Toda columna que entra a un modelo se llama `feat_` o `static_`, y lo de auditoría, `aux_`, el entrenamiento no lo ve. Cada corrida es un YAML versionado con sus métricas en `results/`, más de 200 chequeos automáticos corren antes de cada merge, y cada hallazgo de docs/memoria/ trae el comando que lo reproduce (Anexo A).

5 Expectativas de solución

La solución usa los cuatro campos que pide Ford: **análisis de datos** (§4), **machine learning** (modelos tabulares, de supervivencia y de cura con validación agrupada), **deep learning** (el finalista es una red recurrente con atención) e **IA generativa** (agentes que redactan los avisos, controlados por código).

5.1 Objetivo de mejora: anticipar con el mayor margen posible

5.1.1 El recorrido: más de 40 modelos, la misma cuenta

Con la **primera entrega** (290 autos de desarrollo y dos mercados), los modelos tabulares aprendían *qué auto*, no *cuándo*, y eso nos hizo cambiar la métrica (§3.2). Survival stacking (un hazard por tramo de km) fue el único con un aporte del *cuándo* positivo y estable, y su versión con la venta-

na del registro (K2) fue finalista. Descartamos con evidencia el proceso gamma (no hay desgaste irreversible), el horizonte ordinal, TimesFM, el cure model por hito post-venta, el efecto aleatorio por auto y varios ensambles. **Con la entrega v2** (446 autos y tres veces más eventos), las redes que leen la secuencia pasaron adelante (tabla 3).

Tabla 3: Lo que decidió el finalista, sobre la entrega v2. Detección (%) al 5 · 10 · 20 % de falsas alarmas.

Modelo o idea	Resultado	Qué aprendimos
Re-medición de 42 modelos [dev]	GRU + viajes + estática ×3: 27 · 42 · 61; survival stacking: 14 · 24 · 42; celda: 18 · 33 · 62	Los secuenciales le ganan a survival stacking con evidencia al 10–20 %; ninguno le gana con evidencia a la celda
Sweep bayesiano de la GRU (130 trials) [dev]	El mejor empata con semillas nuevas (0,469 contra 0,471)	La semilla mueve más que los hiperparámetros: se reporta un ensamble de 3
10 semillas, ensambles heterogéneos, MiniRocket [dev]	Ninguno mejora al 5–10 %; MiniRocket pierde (–16 al 10 %, IC95 [–26; –8])	Hay un techo de información, no de modelo
GRU con etiqueta suave lejos del evento	+9,6 puntos en dev (IC95 [3,0; 14,4]); –13,3 en test (IC95 [–25,0; +3,9])	Ganaba en dev y no se sostuvo (\$5.1.5)

Fuente: f9-remedicion-completa-v2.md, f10-sweep-gru.md, f12-preregistro-deteccion.md, f11-*.md; MiniRocket: origin/exp/minirocket:docs/memoria/f13-preregistro-minirocket.md. Todos los modelos en el Anexo C.

5.1.2 El finalista: una red que lee los viajes en orden y recuerda

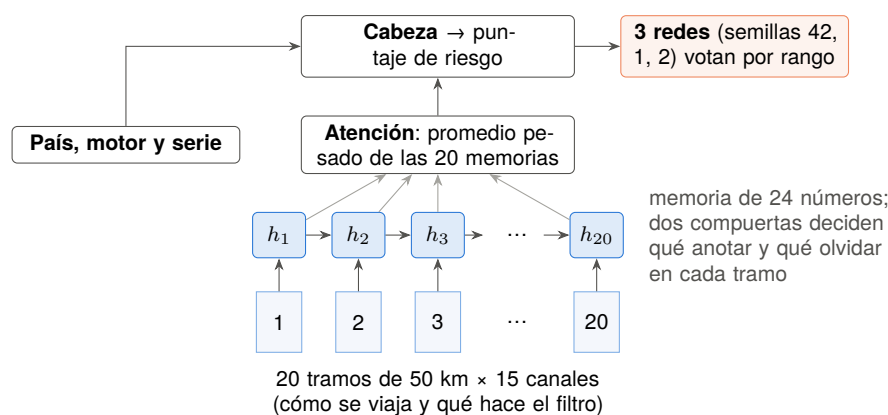


Figura 5: La GRU finalista: ~3.900 parámetros (GRU 2.952, atención 416, estática 52, cabeza 481).

Fuente: src/models/gru_seq.py; configs/exp_v2all_gru_trips_estaticas_r3.yaml.

- **Qué lee.** Cada 500 km, los últimos 1.000 km en 20 tramos. Cada tramo trae 6 señales de cómo se viaja (cantidad de viajes, ralentí, motor bajo régimen, velocidad, km y duración por viaje) y 9 del filtro (acumulación media, máxima y cuánto sube, regeneraciones, avisos y cobertura).
- **Cómo recuerda.** Una GRU unidireccional actualiza su memoria tramo a tramo: la compuerta de actualización decide qué parte se reemplaza con lo nuevo y la de reinicio, cuánto del pasado usa. Así ve tendencias que un promedio borra: que el hollín viene subiendo o que los viajes se acortan. Una capa de atención pesa los tramos, y se suman país, motor y serie.
- **Cómo aprende.** Con la etiqueta dura sobre las ~11.700 filas de desarrollo, pesando más los positivos: Adam, 40 épocas fijas y dropout de 0,3, en CPU. Es chica a propósito: con 135 autos que fallan, una red grande memoriza autos en vez de hábitos. Tres semillas (42, 1 y 2, las del reporte v2, no elegidas por resultado) votan por rango.

5.1.3 El resultado, en autos que nunca vio

El test son los 111 autos del holdout (103 con datos: 32 fallados y 71 sanos). Cada modelo se entrenó con todo el desarrollo y se midió una vez con la misma cuenta que en dev.

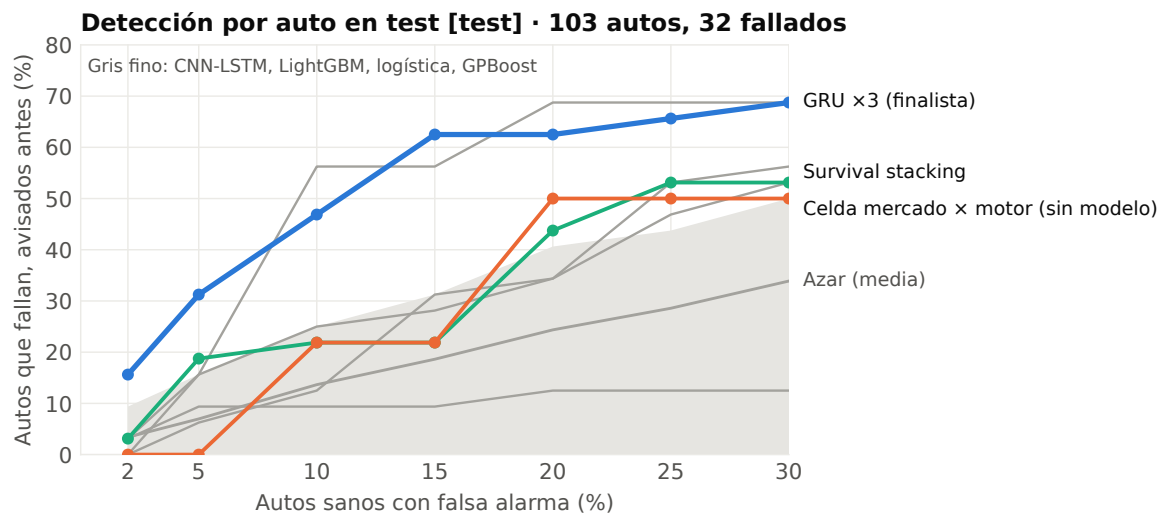


Figura 6: Detección de autos que fallan según el porcentaje de sanos con falsa alarma, en test. La banda gris es el azar hasta su p95. La celda no detecta nada por debajo del 10 % porque alerta celdas enteras.

Fuente: experiments/test-modelos-v2/curve.csv; figura: scripts/make_report_figures.py.

Tabla 4: El finalista en test [test]: detección (%) por presupuesto de falsas alarmas; “Prim.” es la media al 5–20 %.

	5 %	10 %	15 %	20 %	Prim.
GRU x3 (semillas 42, 1, 2)	31,2	46,9	62,5	62,5	50,8
Survival stacking (entrega 1)	18,8	21,9	21,9	43,8	26,6
Celda mercado x motor (sin modelo)	0,0	21,9	21,9	50,0	23,4
Azar: media / p95	7,0 / 15,6	13,7 / 25,0	18,6 / 31,2	24,4 / 40,6	—

Fuente: experiments/test-f10-s42/report.json, experiments/test-modelos-v2/report.json.

- **10 de 32 al 5 %:** 4,5 veces el azar y por encima de su p95. Al 20 %, seis de cada diez.
- **Le duplica el primario a los tabulares y a survival stacking** (50,8 contra 21–27; contra survival stacking, +24,2 puntos, IC95 [0,8; 43,0], comparación no preregistrada). **Contra la celda, +27,3, IC95 [–0,8; 46,1]:** queda en el límite y no la citamos como victoria.
- **El número para operar:** con el umbral fijado antes con los sanos de desarrollo, detecta 20 · 34 · 57 % al 5 · 10 · 20 % con 5,8 · 8,9 · 18,2 % de falsas alarmas reales. El umbral cumple su presupuesto en autos nuevos.
- **Repitió dev** (27,4 · 42,5 · 53,3 · 60,7 %, primario 46,0) y ordena autos con AUC 0,794; dentro de mercado x motor, 0,675.

Por qué un rango. La misma configuración se leyó tres veces en test: 34, 28 y 31 % al 5 %. La primera lectura fue antes del preregistro de F11, y la GRU no es reproducible bit a bit entre corridas. Con 32 fallas cada auto mueve 3 puntos, así que citamos ~30 % al 5 %, ~40–50 % al 10 % y ~50–60 % al 20 %. Sin los 36 autos de test que estaban en el desarrollo de la primera entrega da 36,4 · 50,0 · 59,1 % al 5 · 10 · 20 %: lo aprendido con la primera entrega no infla el test.

5.1.4 Cuánto antes: el margen medido

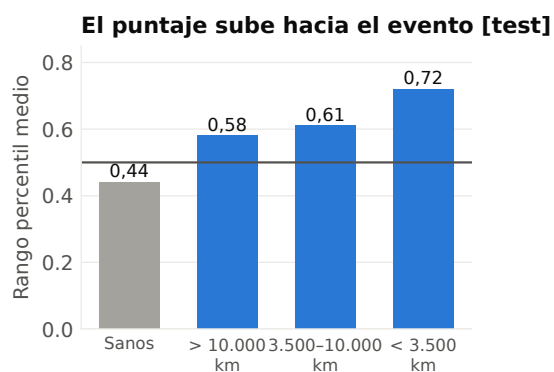


Figura 7: Rango percentil medio del puntaje en los sanos y en los cortes de autos que fallan, según la distancia al evento [test].

Fuente: f11-test-resultado.md §4.

- **Al 10 % [test]:** mediana de 4.646 km (p25–p75: 2.403–9.263 km) entre la primera alerta y el evento, unos **99 días**. Al 5 %: 3.987 km (~72 días); al 20 %: 5.969 km (~109 días).
- **En dev [dev]:** ~7.600 km (~86 días). Los autos detectados en test andan menos km por día: en días el margen es parecido.
- **El piso es el gap:** ninguna alerta usa los 500 km previos. Se mide hasta la referencia del evento (fecha registrada – 21 días).
- **Margen, no pronóstico:** el puntaje sube hacia el evento, pero la dispersión es enorme. Decimos “lo marcamos con unos tres meses de margen”, nunca “falla en tres meses”.

5.1.5 Una variante que ganaba en desarrollo y no se sostuvo

Con la etiqueta dura, un corte lejano de un auto que va a fallar se entrena como si fuera sano. La variante F11 le ponía 0,15 a esos cortes. En una confirmación preregistrada en dev ganaba con evidencia (+9,6 puntos, IC95 [3,0; 14,4]) y era el primer modelo que le ganaba a la celda. En el tiro preregistrado en test dio 31,2 contra 44,5 (–13,3, IC95 [–25,0; +3,9]): se comportó como la tasa de la celda y perdió el orden dentro de ella (AUC 0,689 en dev, 0,571 en test). Lo reportamos tal cual, sin buscar variantes. La lección: la confirmación usó folds y semillas nuevos, pero los mismos 446 autos sobre los que se habían explorado unas 20 variantes. El finalista es la GRU con la etiqueta dura porque era la referencia preregistrada, no porque se haya elegido mirando test.

Límites

1. **No le gana con evidencia a la celda mercado × motor [test]:** parte de lo que detecta es composición, y el país pesa (§5.2.5).
2. **Sabe poco del cuándo:** (a') = +0,001 [dev]; colapsar el puntaje al promedio del auto casi no cambia nada.
3. **Con 32 fallas los intervalos son anchos:** sin los autos vistos, survival stacking y LightGBM empatan con la GRU.
4. **El test se leyó más de una vez:** ninguna variante nueva se mide contra él; mejorar pide datos nuevos.
5. **La muestra está enriquecida en fallas:** el puntaje ordena autos y no es una probabilidad calibrada a la prevalencia real. Tampoco predecimos la fecha exacta.

5.2 Criterios de evaluación

5.2.1 Viabilidad técnica

Lo que ya existe: un modelo medido en un holdout con su umbral fijado fuera de muestra, un pipeline reproducible de los CSV al puntaje y una demo desplegada. **Por qué escala:** cero sensores nuevos (solo resúmenes de viaje y mensajes del filtro que el auto ya manda); una GRU de ~3.900 parámetros que corre en CPU (las 56 corridas de la re-medición tardaron ~25 minutos en 24 núcleos); el mismo código de features en entrenamiento y en producción; y agentes que trabajan solo sobre las alertas. Para un millón de autos estimamos un batch diario de 1–2 horas en 8–16 vCPU, a validar con Ford.

Tabla 5: Cómo llega a producción.

Etapa	Qué	Duración	Para pasar
0 · Congelar	Reentrenar con los 557 autos y empaquetar (pesos, ONNX, referencia del ensamble, umbrales)	1–2 semanas	Re-puntúa igual 20 autos de control
1 · Sombra	Puntuar la flota real de Colombia o Chile sin avisar a nadie	3–6 meses	Alertas $\approx$ presupuesto; detección $\geq$ celda
2 · Piloto activo	Avisos en una región, con grupo control	6 meses	Fallas evitadas; costo por alerta; hábitos
3 · Escala	Mercados validados, umbral por mercado	continuo	Monitoreo en verde

Fuente: docs/pitch/presentacion-contenido.md bloque 5.

La sombra es el paso clave: es la primera medición de la prevalencia real y de las falsas alarmas sobre la flota completa, no sobre las listas de Ford. El monitoreo sigue la tasa de alertas por mercado, la deriva de las features, la detección realizada y los datos que dejan de llegar (ya pasó con el contador de regeneraciones). El reentrenamiento es trimestral, con las mismas auditorías. Falta escribir el empaquetado del modelo (`export_model.py`); no toca nada de lo medido.

La **demo** reproduce semana a semana los 426 autos de desarrollo con el modelo finalista (el test no aparece nunca) en tres pantallas: la bandeja del gerente de posventa, la ficha del auto para el concesionario y “qué pasó después”. Una perilla elige el punto de operación. Al 5 % [dev]: 53 alertas y 7 escalamientos; 39 de los 135 autos que fallan reciben aviso, con una mediana de 15 semanas (~ 7.000 km) de margen; 14 de 291 sanos reciben uno de más; 48 de las 53 alertas llevan un hábito para nombrar. Al 20 %: 82 de 135 avisados y 58 sanos con aviso de más. Dos agentes redactan (el redactor y el de triage, que no puede elegir otra acción que la de la política) y un **verificador en código** rechaza números que no estén en los hechos, lenguaje causal, certezas, promesas y síntomas del filtro en un texto al conductor. En toda la temporada redactaron 465 textos y 196 resúmenes, todos aprobados, con 40 intentos rechazados y ninguna plantilla.

5.2.2 Costo

Tabla 6: Costo anual de operación para 1 millón de autos (estimación propia, a validar con Ford).

Componente	USD por año	Supuesto
Features + puntaje diario	2.500–6.000	batch de 1–2 h en 8–16 vCPU
Agentes (LLM)	1.000–3.000	~ 65.000 alertas por año, modelo chico
Reentrenamiento + monitoreo	5.000–10.000	trimestral, en CPU
Equipo de operación	60.000–80.000	1 persona de ML/MLOps
Total	~ 70.000–100.000	$\sim$USD 0,07–0,10 por auto y por año

Fuente: docs/pitch/presentacion-contenido.md bloque 6. Implementación: $\sim$ USD 70.000–110.000 una vez.

El 80 % del costo es la persona que lo opera: sumar autos casi no lo mueve. No citamos un ahorro porque no conocemos lo que le cuesta a Ford una falla: con escenarios de precios públicos, el modelo paga cuando anticipar vale entre ~ 5 y ~ 15 veces una inspección y la prevalencia real es de 2–5 % (`f8-costos-k2.md`). El diseño baja el costo de equivocarse: el primer contacto es un consejo, no un turno. Es necesario porque en una flota real la mayoría de los avisos irán a autos que no iban a fallar (con 5 % de prevalencia y la perilla al 5 %, unos 2 de cada 10 avisos son de un auto que iba a fallar). La pérdida de consumo no se pudo medir con estos datos: es la primera métrica del piloto.

5.2.3 Carácter innovador

1. **Una métrica que no se deja engañar:** detección por auto con presupuesto de falsas alarmas, contra un azar que conserva el historial y contra la regla “país $\times$ motor”, en vez de un PR-AUC que se gana sin anticipar.

2. **Una red que lee la ventana en orden** y le duplica la detección a los modelos tabulares en test.
3. **IA generativa con barandas:** el modelo decide, el agente redacta y el código verifica; el LLM nunca produce un número ni un diagnóstico.
4. **El porqué como comparación, no como causa, y la perilla** en manos de Ford.
5. **Rigor como producto:** preregistros, auditorías obligatorias y un holdout que se abrió para medir, no para elegir.

5.2.4 Claridad para mostrar los datos, y el pitch

Cada figura de este informe responde una sola pregunta, trae su fuente y sale de un script con su YAML. La central es la curva de detección contra falsas alarmas con el azar dibujado (figura 6). La demo muestra el calendario de la flota, la ficha del auto con su puntaje contra el umbral y los hábitos contra la flota sana, y la perilla. Hay además un dashboard del análisis exploratorio (solo desarrollo). El pitch cuenta la misma historia en siete bloques y 29 minutos, con los mismos números y las mismas reglas: toda detección al lado del azar, el resultado en test y en rango, la anticipación como margen y los límites dichos antes de que los pregunten ([docs/pitch/](#)).

5.2.5 Explicabilidad del modelo

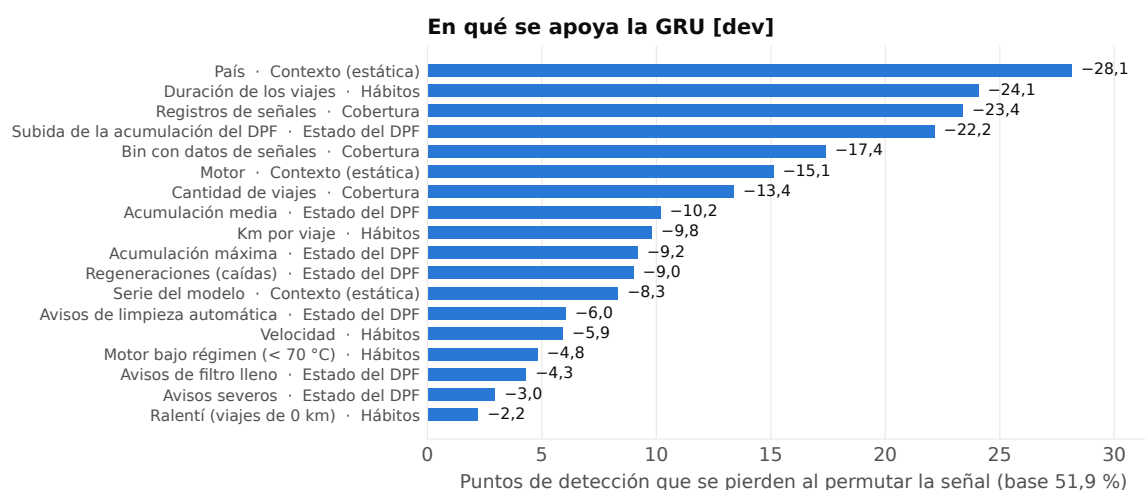


Figura 8: Puntos de detección (media al 5–20 %) que se pierden al permutar cada señal en la validación [dev], sobre una base de 51,9 %. Las caídas no se suman por familia.

Fuente: `experiments/explain-gru-final/units.csv`; `configs/explain_gru_final.yaml`.

En qué se apoya la GRU (figura 8). El hábito que más pesa es la duración de los viajes (–24,1 puntos), por delante de km por viaje, velocidad, motor bajo régimen y ralentí. Del filtro, cuánto sube la acumulación (–22,2). El país pesa más que cualquier señal (–28,1), coherente con la celda como piso. Es la lectura física del DPF: viajes cortos que no dejan terminar la regeneración y hollín que se acumula. No le dimos ninguna regla del filtro.

Qué se le dice al conductor. La GRU no tiene atribución por auto, y 7 de sus 15 canales son el estado del propio filtro, que el conductor no puede cambiar. Por eso el porqué que se comunica es una comparación: en qué hábitos el auto supera al 75 % de los sanos de su mercado, hasta tres, y solo entre los accionables cuyo efecto coincide con la física del DPF (con el finalista de la entrega 1, 10 de 15 hábitos con hipótesis física coincidían y cuatro iban al revés: esos no se nombran). Síntomas y contexto van solo al taller. El texto dice “comparado con autos sanos de tu zona”, nunca “porque”.

6 Información complementaria

Documentación y referentes. Recibimos el dataset y el diccionario en dos entregas, y contrastamos el diccionario contra los archivos columna por columna (§4). Todo lo que encontramos quedó en el

repositorio: un archivo por hallazgo con su comando en `docs/memoria/`, cada decisión con su fecha y su motivo en `decisiones.md`, y cada corrida como YAML en `configs/`. La CNN-LSTM propuesta por la tutora del equipo fue el primer baseline secuencial y una de las familias medidas en test.

Lo que solo Ford tiene. Hoy el límite no es el modelo, es la información:

1. **La tasa real de falla por país × motor** en la flota. Separa la física de cómo se armaron las listas, y es la duda más grande del proyecto.
2. **Qué eran los 278 fallados con fecha por defecto** que salieron entre entregas, y con qué criterio se armó la lista de fallados.
3. **Qué registra la fecha del evento:** hoy la corremos 21 días porque cae unas dos semanas después del taller.
4. **Más fallas registradas** (hoy son 177), **dónde se usa el auto** (no dónde se vendió) y **los costos reales** de una falla y de una inspección.
5. **Un piloto en sombra de 3 meses en un país**, para medir la prevalencia, las falsas alarmas y la detección reales.

Trabajo futuro. Reentrenar con los 557 autos y empaquetar el modelo. Validar cualquier mejora con datos nuevos (una extracción posterior al 14-09-2026), no con este test. Medir en el piloto lo que estos datos no permiten: si los avisos cambian hábitos, cuántas fallas se evitan y cuánto consumo se ahorra. Extender el circuito a otras fallas del postratamiento.

Conclusión. La telemetría que Ford ya recibe anticipa la falla del DPF, pero leerla bien fue la mitad del trabajo, y medir bien la otra mitad. La GRU finalista avisa a ~30 % de las fallas con 5 % de falsas alarmas y a ~50–60 % con 20 %, con unos tres meses de margen, en autos que nunca vio. Le duplica la detección a los modelos tabulares y de supervivencia, sin ganarle con evidencia a la celda. Y el producto convierte esa alerta en un consejo confiable, con la perilla en manos de Ford. *Del testigo en el tablero al aviso a tiempo: tu Ford te avisa antes.*

A Reproducibilidad

Todo corre desde la raíz del repositorio (`requirements.txt` y `requirements-dl.txt`); la entrega v2 va en `data/v2/raw/`. La GRU no es reproducible bit a bit entre plataformas: una reproducción da números cercanos, no idénticos. Cada comando lee su YAML de `configs/`.

```
export FORD_DATA_DIR=$PWD/data/v2 WANDB_MODE=disabled OMP_NUM_THREADS=1
# universo, holdout, EDA y paneles
python scripts/make_test_split.py --config configs/data/test_split.yaml --force
python scripts/eda_v2.py --config configs/eda_v2.yaml
python scripts/build_dataset.py --config configs/data/panel_v2_estaticas.yaml
python scripts/make_splits.py --config configs/data/splits_panel_v2_r3.yaml
python scripts/build_seq_panel.py --config configs/data/panel_seq_trips_v2_estaticas.yaml
# el finalista en dev: una corrida por semilla (y _s1_r3, _s2_r3), ensamble, auditorías, reporte,
  ↳ explicabilidad
python scripts/train.py --config configs/exp_v2all_gru_trips_estaticas_r3.yaml
python scripts/ensemble_rank.py --config configs/exp_v2all_seeds3_gru_trips_estaticas.yaml
python scripts/audit_model.py --config configs/exp_v2all_gru_trips_estaticas_r3.yaml
python scripts/report_v2_models.py --config configs/report_v2_models.yaml
python scripts/explain_perm_seq.py --config configs/explain_gru_final.yaml
# la medición en test (ya hecha: el test no se vuelve a usar para elegir)
python scripts/eval_test.py --config configs/eval_test_f10_s42.yaml --report
python scripts/eval_test.py --config configs/eval_test_modelos_v2.yaml --report
# este informe
python scripts/make_report_figures.py --config configs/report_figures.yaml
cd docs/informe && tectonic informe.tex
```

B Auditorías de leakage y el techo de cohorte

Además de las defensas de construcción (gap de 500 km sobre la referencia corrida, split por vehículo con los clones colapsados, features hacia atrás y corte en el fin de extracción de los sanos), toda corrida candidata pasa por `scripts/audit_model.py`: (**a0**), que permuta las features entre to-

das las filas, reentrena y exige que el PR-AUC caiga a la tasa base (si no cae, hay leakage); (a'), que colapsa el puntaje de cada auto a su promedio sin reentrenar (lo que cae es lo que el modelo sabía del *cuándo*); y (b), que le da al modelo las columnas de calendario y marca un atajo si el ROC sube más de 0,02. La permutación dentro del vehículo no es un nulo: conserva qué autos fallan y sube en vez de bajar.

Tabla 7: Auditorías de la entrega v2 [dev] (semilla 42 de cada secuencial; tasa base 0,058).

Modelo	(a0) PR-AUC	(a')	(b) calendario	Veredicto
GRU + viajes + estática (finalista)	0,061	+0,001	+0,004	aprueba
GRU + viajes	0,061	+0,003	+0,025: marca	aprueba
CNN-LSTM + viajes + estática	0,057	+0,009	+0,018	aprueba
CNN-LSTM de la tutora (señales + país)	0,056	+0,026	+0,028: marca	aprueba
Survival stacking + motor y serie	0,056	+0,025	+0,009	aprueba

Fuente: docs/memoria/f9-remedicion-completa-v2.md.

Tabla 8: PR-AUC por fila en el dev v2 [dev], al lado del techo de cohorte.

	PR-AUC (lift)	(a')
Tasa base	0,057 (0,97×)	−0,002
LightGBM (control)	0,112 (1,92×)	−0,019
Survival stacking	0,125 (2,14×)	+0,016
GRU + viajes + estática ×3	0,160	+0,006
Techo de cohorte ("¿este auto falla?", sin nada del cuándo)	0,177 (3,03×)	—

Fuente: f9-remedicion-v2.md; f9-remedicion-completa-v2.md (la (a') del ensamble es +0,006; la de la semilla 42, +0,001).

Ningún modelo filtra: con las features permutadas, todos caen a la tasa base. Ninguno supera el techo de cohorte, y la (a') es positiva pero chica en todos: lo que separa es sobre todo *qué auto*. El finalista no marca el atajo de calendario, mientras que la misma GRU sin estática sí lo marcaba. Con la primera entrega, además, el tamaño de la bolsa solo (18 cortes por fallado contra 9 por sano) daba lift 1,79×: desde entonces toda detección se compara contra un nulo que conserva el largo del historial.

C Todos los modelos en test y en desarrollo

Cada modelo se entrenó con todo el desarrollo (semilla 42, salvo los ×3) y se midió con la misma cuenta en los 103 autos de test. Se midieron después del tiro preregistrado: describen, no eligen.

Tabla 9: Detección (%) en test por presupuesto de falsas alarmas [test]. "Prim." es la media al 5–20 %.

Modelo	2 %	5 %	10 %	15 %	20 %	25 %	30 %	Prim.	AUC auto	AUC celda	dev (5 · 10 · 20 %)
GRU ×3 (finalista)	15,6	31,2	46,9	62,5	62,5	65,6	68,8	50,8	0,794	0,675	27 · 42 · 61
CNN-LSTM ×3	3,1	15,6	56,2	56,2	68,8	68,8	68,8	49,2	0,790	0,638	22 · 41 · 62
Survival stacking	3,1	18,8	21,9	21,9	43,8	53,1	53,1	26,6	0,710	0,515	14 · 24 · 42
LightGBM (53 features)	0,0	15,6	25,0	28,1	34,4	53,1	56,2	25,8	0,725	0,619	18 · 28 · 42
Logística L2 (53 features)	0,0	6,2	12,5	31,2	34,4	46,9	53,1	21,1	0,679	0,556	15 · 23 · 37
SS + efecto aleatorio (GPBoost)	3,1	9,4	9,4	9,4	12,5	12,5	12,5	10,2	0,363	0,377	6 · 9 · 17
Celda mercado × motor (sin modelo)	0,0	0,0	21,9	21,9	50,0	50,0	50,0	23,4	—	—	18 · 33 · 62
Azar, media / p95 (nulo de la GRU)	3,4 / 9,4	7,0 / 15,6	13,7 / 25,0	18,6 / 31,2	24,4 / 40,6	28,6 / 43,8	33,9 / 50,0	—	—	—	—

Fuente: Detección y azar: experiments/test-modelos-v2/curve.csv (tabla generada); AUC y dev: docs/memoria/f11-test-resultado.md §3.

Comparaciones pareadas en test (bootstrap por vehículo, diferencia de primario): GRU – celda, +27,3 [−0,8; 46,1]; survival stacking – GRU, −24,2 [−43,0; −0,8]; GRU 42/1/2 – GRU 101–103, +6,2 [−7,8; +18,0]; GRU suave – GRU dura (tiro preregistrado de F11), −13,3 [−25,0; +3,9].

Tabla 10: Con el umbral fijado en dev [\[test\]](#): detección y falsas alarmas reales en test (15 modelos de fold).

Modelo	Detección (%) al 5 · 10 · 20 %	Falsas alarmas reales (%)
GRU ×3 (finalista)	20 · 34 · 57	5,8 · 8,9 · 18,2
CNN-LSTM ×3	16 · 34 · 55	5,6 · 8,4 · 14,8
Survival stacking	13 · 20 · 33	4,6 · 8,3 · 15,6
LightGBM (53 features)	16 · 25 · 36	7,5 · 10,7 · 16,9
Logística L2 (53 features)	12 · 16 · 33	8,8 · 11,8 · 18,3
SS + efecto aleatorio (GPBoost)	8 · 11 · 14	6,5 · 11,3 · 21,5

Fuente: experiments/test-modelos-v2/operating.csv (tabla generada).

Tabla 11: Los modelos principales en el dev v2 [\[dev\]](#): detección (%) con umbral exacto, promedio de 3 repeticiones.

Modelo	AUC auto	AUC celda	5 %	10 %	20 %
GRU + viajes + estática ×3 (finalista)	0,82	0,65	27	42	61
CNN-LSTM + viajes + estática ×3	0,81	0,62	22	41	62
CNN-LSTM de la tutora + estática ×3	0,82	0,62	27	40	68
Survival stacking (F3)	0,72	0,57	14	24	42
LightGBM (control, 53 features)	0,72	0,57	18	28	42
Logística L2	0,71	0,58	15	23	37
Celda mercado × motor (sin modelo)	0,81	0,46	18	33	62
Azar (historial conservado)	—	—	5	11	21

Fuente: docs/memoria/f9-remedicion-completa-v2.md (42 modelos).

D Diccionario de features

Las 53 features agregadas se calculan sobre la ventana (corte – 1.000 km, corte], solo con datos anteriores al corte, como `agregador(columna derivada)`. Las `aux_` quedan para auditar y no entran a ningún modelo. Agregadores: `per_1000km`, conteo por 1.000 km; `per_day`, por día; `half_*`, segunda mitad de la ventana menos la primera (tendencia); `gap_*`, distancia entre eventos; `any`, si ocurrió alguna vez.

A · Térmica y trayectos cortos

```
idle_frac: mean(idle)
idle_per_1000km: per_1000km(idle)
idle_frac_trend: half_mean_delta(idle)
short_trip_frac_5km: mean(short_trip_moving)
trips_below_regime_temp_frac: mean(below_regime)
below_regime_moving_frac: mean(below_regime_moving)
below_regime_frac_trend: half_mean_delta(below_regime)
engine_temp_avg_median:
median(EngineTemperatureAvg_moving)
engine_temp_max_median:
median(EngineTemperatureMax_moving)
engine_temp_amplitude_mean: mean(engine_temp_amplitude_moving)
coolant_temp_end_mean: mean(CoolantTemperatureEnd_moving)
coolant_temp_start_median:
median(CoolantTemperatureStart)
cold_start_frac: mean(cold_start)
chained_trip_frac: mean(chained)
trip_distance_median_km: median(trip_km_moving)
trip_distance_p25_km: p25(trip_km_moving)
trip_duration_median_min: median(trip_duration_min_moving)
```

B · Regeneración (DPF)

```
regenerations_per_1000km: per_1000km(regen_drop)
regenerations_trend_per_1000km: half_count_delta_per_
```

```
1000km(regen_drop)
distance_between_regen_mean_km: gap_mean_km(regen_drop)
distance_between_regen_trend: gap_trend_km(regen_drop)
km_since_last_regen: km_since_last(regen_drop)
regen_residual_mean: mean(regen_residual)
regen_start_level_mean: mean(regen_start_level)
dpf_end_mean: mean(AirRegenerationEnd)
dpf_end_max: max(AirRegenerationEnd)
dpf_end_slope_per_1000km: half_slope_per_1000km(AirRegenerationEnd)
dpf_positive_delta_frac: mean(dpf_positive_delta_moving)
dpf_saturated_frac: mean(dpf_saturated_end)
filter_abnormal_end_frac: mean(filter_abnormal_end)
filter_cleaning_auto_end_frac: mean(filter_cleaning_auto_end)
manual_regen_ever: any(filter_manual)
```

C · Uso

```
speed_kmh_mean: mean(speed_kmh_moving)
trips_below_30kmh_frac: mean(urban_moving)
moving_trips_per_1000km: per_1000km(moving)
km_per_day: per_day(trip_km)
trips_per_day: per_day(moving)
hours_between_trips_median: median(soak_min)
aux_air_temp_avg: mean(AirTemperatureAvg)
aux_air_temp_min: mean(AirTemperatureMin)
```

D · Severidad (mensajes, aceite, consumo)

```

msg_full_per_1000km: per_1000km(msg_full)
msg_overloaded_per_1000km: per_1000km(msg_overloaded)
msg_overloaded_ever: any(msg_overloaded)
msg_over_limit_per_1000km: per_1000km(msg_over_limit)
msg_over_limit_ever: any(msg_over_limit)
msg_at_limit_ever: any(msg_at_limit)
msg_cleaning_auto_per_1000km: per_1000km(msg_cleaning_auto)
msg_cleaning_auto_trend_per_1000km: half_count_delta_per_1000km(msg_cleaning_auto)

```

```

msg_abnormal_frac: mean(msg_abnormal)
msgs_per_1000km: per_1000km(msg_any)
oil_life_mean: mean(oil_life)
oil_life_slope_per_1000km: half_slope_per_1000km(oil_life)
fuel_pct_per_100km_median: median(fuel_pct_per_100km)
aux_regen_marker_per_1000km: per_1000km(regen_marker)
Control de ventana
n_trips_window: count(*)
window_km_covered: rango(OdometerTripEnd)

```

Fuente: configs/data/features_v1.yaml (lista generada); derivadas en src/features/trips.py y src/features/signals.py.

Los 15 canales de la GRU. La ventana se divide en 20 tramos de 50 km; “zero” rellena con cero un tramo sin datos y “carry” arrastra el último valor dentro de la misma ventana. Más país, motor y serie en one-hot.

Canal	Fuente	Columna	Por tramo	Relleno
n_records	signals	*	count	zero · log1p
observed	signals	*	any	zero
acum_mean	signals	Acumulation	mean	carry
acum_max	signals	Acumulation	max	carry
acum_rise	signals	acum_rise	sum	zero
regen_drops	signals	acum_regen_drop	sum	zero
msg_full_frac	signals	msg_full	mean	zero
msg_severe_frac	signals	msg_severe	mean	zero
msg_cleaning_auto_frac	signals	msg_cleaning_auto	mean	zero
n_trips	trips	*	count	zero · log1p
idle_frac	trips	idle	mean	carry
below_regime_frac	trips	below_regime_moving	mean	carry
speed_kmh	trips	speed_kmh_moving	mean	carry
trip_km	trips	trip_km_moving	mean	carry
trip_duration_min	trips	trip_duration_min_moving	mean	carry

Fuente: configs/data/panel_seq_trips_v2_estaticas.yaml (tabla generada).